

Universitat Autònoma de Barcelona
Master Degree in Modelling for Science and Engineering
Institution / Lab: IIIA-CSIC

Technical Memory Draft Baseline

End-to-End Planner Dialogue Flow: Controlled Revision

Formal architecture and contract section for technical review.

Document type:	Architecture and contract walkthrough
Academic year:	2025–2026
Date:	2026-06-09
Scope:	Runtime flow, ROS contracts, compact grounding, planner output, feedback, and dialogue ownership.
Repository:	https://github.com/ieverythng/nao-ros4hri-bridge

1 Purpose

This document is the technical and thesis-facing walkthrough for the active planner stack. It describes the path from a user utterance to robot execution and spoken feedback, with emphasis on which node owns each decision. The document is payload-oriented to highlight the importance of the contracts in the end-to-end planner dialogue flow.

The chatbot extracts the user's intent and converts it into a goal; the planner proposes a structured plan over known skills; the orchestrator validates and dispatches executable skills; skills execute the actions with fresh live evidence before executing and reporting outcomes; lastly, the dialogue manager handles the verbal interaction with the user by using an LLM-based chatbot.

A *turn* is one dialogue exchange initiated by a user utterance and processed by the dialogue stack. Turns may remain conversational including pure dialogue (no execution of skills) or knowing about the robots internal representation of the world. Turns may also produce a request to the robot to plan and execute a task based on the skills available.

2 End-to-End Runtime Flow

Figure 1 shows the runtime message sequence. Planning and execution are separate, raw world evidence is only exposed to skills while planner and chatbot use a simplified JSON payload, and speech is realized by the dialogue layer.

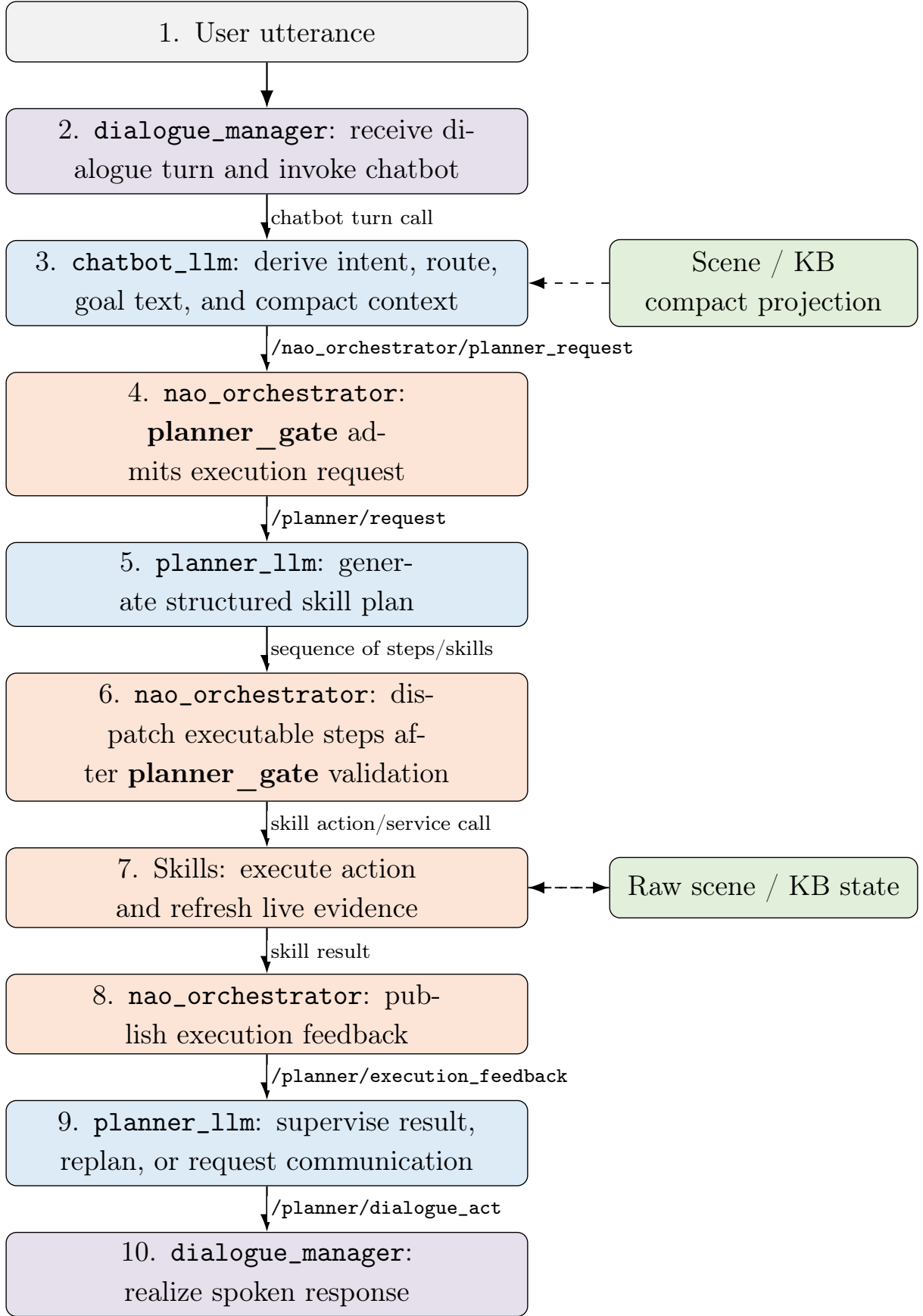


Figure 1: Runtime message and responsibility sequence from user input to spoken response. The dialogue manager owns the turn and speech realization; `chatbot_llm` derives the intent, route, goal text, and compact context; `nao_orchestrator` admits and dispatches execution; `planner_llm` generates and supervises the skill plan; and skills refresh live evidence before reporting outcomes.

Its important to note that the **planner_gate** is an admission stage inside **nao_orchestrator**; it is not a second planner. Its role is to receive a candidate execution request from the chatbot path, reject non-execution turns, normalize fields such as **request_kind**, attach or validate goal metadata, publish the admitted task on **/planner/request** and decline or accept plans for execution.

3 Runtime Ownership Rules

- **dialogue_manager** owns the dialogue turn lifecycle and user-facing speech realization.
- **chatbot_llm** owns user-facing semantic interpretation: intent derivation, route selection (dialogue or execution) and goal text for planner.
- **nao_orchestrator** owns deterministic planner admission, request normalization, executable-step validation, dispatch, step status, and feedback publication.
- **planner_llm** owns structured skill-plan generation, plan supervision, replanning decisions, and dialogue-act intent.
- Skills own live interaction with the robot, scene, and KB evidence required to complete their operation truthfully.

4 Contract Map and Field Presentation

Table 1 lists the active runtime contracts.

Table 1: Runtime contract map and ownership in the active planner stack.

ID	Contract	Interface / payload	Ownership
1	Chatbot turn output	Internal chatbot result before ROS planner publication	dialogue_manager invokes; chatbot_llm produces semantic result
2	Planner request and admission	Candidate /nao_orchestrator/planner_request admitted as /planner/request ; payload in Intent.data	chatbot_llm proposes; nao_orchestrator admits/normalizes; planner_llm consumes
3	Compact grounding	grounded_context inside Contract 2	Scene/KB projection for chatbot_llm and planner_llm
4	Planner output	/intents ; payload in Intent.data.plan	planner_llm produces; nao_orchestrator validates and dispatches
5	Execution feedback	/planner/execution_feedback ; JSON payload	nao_orchestrator publishes; planner_llm supervises
6	Planner dialogue act	/planner/dialogue_act ; structured communication request	planner_llm emits; dialogue_manager realizes speech
7	Skill execution	ROS skill actions/services	nao_orchestrator dispatches; skill servers execute
8	Raw evidence	/scene/summary , /kb/query , knowledge snapshots, action results	Scene/KB/skills provide evidence for compact projection and skill-local checks

The table above lists only the contract and its meaning. Compact planner-facing fields are

kept in the tables, while additional runtime metadata is described in prose when it is relevant for orchestration, planner supervision, tracing, or debugging.

5 Contract 1: Chatbot Turn Output

The intent in this contract is the user’s goal as extracted by the chatbot. It is not a planner output.

```
{
  "verbal_ack": "Okay, I will do that.",
  "route": "execution",
  "confidence": 0.82,
  "user_intent": {
    "type": "look_at",
    "goal_text": "look at the person",
    "scene_targets": ["person"],
    "request_kind": "new_goal"
  }
}
```

Table 2: Fields in Contract A.

Field	Meaning
<code>verbal_ack</code>	Optional immediate acknowledgement generated by the chatbot layer.
<code>route</code>	Classification as dialogue, knowledge_query , or execution. Execution defines planner path with dialogue and knowledge_query for trace logging.
<code>confidence</code>	Confidence assigned by the chatbot/intention extraction stage; not a planner priority.
<code>user_intent.type</code>	Normalized user-intent label extracted by the chatbot.
<code>user_intent.goal_text</code>	Natural-language objective passed to the planner as the task to satisfy.
<code>user_intent.scene_targets</code>	Optional target hints extracted from the utterance, such as person or cup.
<code>user_intent.request_kind</code>	Proposed goal transition class. The orchestrator gate should normalize/admit this before publishing /planner/request .

6 Contract 2: Planner Request

The planner request is the task description generated by the orchestrator from the chatbot output and consumed by the planner

```
{
  "goal_id": "goal_turn_123",
  "request_kind": "new_goal",
  "goal_text": "look at the person and report completion",
  "normalized_intents": ["look_at"],
  "grounded_context": {
    "entities": [
      {
        "id": "anonymous_person_ehfbf",
        "label": null,
        "kind": "person",
        "class": "Human",
        "visible": true,
        "relations": []
      }
    ]
  }
}
```

Table 3: Fields in Contract B: minimal planner-facing view.

Field	Meaning
<code>goal_id</code>	Identifier for the active objective. It groups replans and feedback for the same task.
<code>request_kind</code>	States whether this is a new goal, an update to the current goal, a cancellation, or another supported transition. <code>chatbot_llm</code> may propose it, but <code>nao_orchestrator</code> normalizes and admits it before publishing <code>/planner/request</code> .
<code>goal_text</code>	Objective extracted from the user utterance. This is the central planning input.
<code>normalized_intents</code>	Intent labels extracted by <code>chatbot_llm</code> . They help <code>planner_llm</code> choose suitable skills from the available skill registry.
<code>grounded_context</code>	Compact scene graph used by the planner as current state of the world.

Implementation metadata can still exist on the ROS envelope or runtime trace. Examples include request identifiers, dialogue turn identifiers, planner mode, parent goal ID, or superseded goal ID. These fields are useful for tracing, debugging, and goal management, but they are not the core task description consumed by `planner_llm`.

7 Contract 3: Grounded Context

The planner receives a compact, JSON-native view of relevant world state. This is best described as a compact scene graph derived from the KB, continually updated by the perception pipeline (human/person detection and object perception). Raw KB triples, detector metadata, and tracker-derived details remain available through scene/KB evidence paths to skills, but they are not duplicated into every LLM prompt.

```
{
  "grounded_context": {
    "entities": [
      {
        "id": "cup_jrjic",
        "label": "cup",
        "kind": "object",
        "class": "Cup",
        "visible": true,
        "relations": [
          {"predicate": "dbp:color", "object": "blue"},
          {"predicate": "oro:isOn", "object": "table_1"}
        ]
      },
      {
        "id": "anonymous_person_ehfbf",
        "label": null,
        "kind": "person",
        "class": "Human",
        "visible": true,
        "relations": []
      }
    ]
  }
}
```

Table 4: Fields in Contract C.

Field	Meaning
id	Stable entity handle used by planner steps and skill arguments.
label	Human-readable name or class hint. It may be empty for anonymous people.
kind	Entity category, currently object or person. This avoids treating people as generic objects.
class	Compact semantic class used by the planner for reasoning.
visible	Whether the entity is currently grounded in perception.
relations	Prioritized semantic relations that add information beyond the class, such as color or containment.

The compact view should omit detector score, image coordinates, backend, source, and re-ency by default. Those values are execution evidence, not planning semantics, unless a future skill explicitly needs them in its own action arguments.

8 Grounded Context Pipeline

JSON object passed (along the system prompt and skills registry) to the chatbot and planner. Acts as LLM facing grounded context representing current entities in the KB.

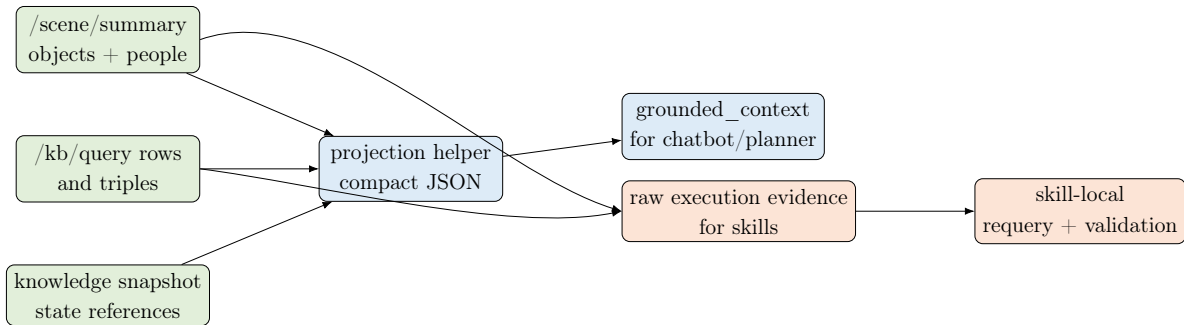


Figure 2: Grounded context is passed from the KB as evidence for LLM-facing reasoning, while raw execution evidence remains available to skill-local requery and validation. The planner request does not need a separate direct tracker input if people already appear in the scene or KB evidence.

Derivation rules:

1. Use text-derived entities only when unable to confirm entities with KB.
2. Keep anonymous people anonymous; do not invent personal names.
3. Keep IDs and labels separate.
4. Bound relation count per entity to keep prompt size manageable.

9 Planner LLM Internals

After the planner request is admitted, `planner_llm` builds a prompt from the compact request, compact grounding, the skill registry, allowed step types, and the output contract. Rule-based shortcuts may produce deterministic plans for simple motions. Otherwise, the LLM provider returns JSON, which is parsed and validated before publication. Invalid JSON or unsupported steps trigger a validation retry or a failure result.

The planner prompt should not contain raw detector coordinates, full RDF dumps, or stale perceptual reports. Those belong to skills and execution evidence.

10 Contract 4: Planner Output

The planner output is a plan: a sequence of skill steps to achieve the user goal. It should focus on executable content and minimal lineage. Communication policy and retry metadata are still valid runtime fields when used by the stack, but this controlled version foregrounds the ordered `steps` list because that is the essential planner output.

```
{
  "plan": {
    "goal_id": "goal_turn_123",
    "plan_id": "plan_123",
    "plan_version": 1,
    "status": "planned",
    "steps": [
      {
        "id": "step_1",
        "type": "skill",
        "name": "look_at",
        "args": {"target_frame": "anonymous_person_ehfbf"},
        "requires": [],
        "on_failure": "replan"
      }
    ]
  }
}
```

Table 5: Fields in Contract D.

Field	Meaning
<code>goal_id</code>	User objective that this plan attempts to satisfy.
<code>plan_id</code>	Identifier for this concrete plan proposal; useful when runtime keeps plan history.
<code>plan_version</code>	Revision number for replans of the same goal.
<code>status</code>	Planning outcome for this plan, including planned , needs_clarification , or failed_to_plan . This describes whether planner_llm produced an executable plan, not whether the robot has already executed it.
<code>steps</code>	Ordered skill calls produced by the planner. This is the essential output.
<code>steps[].id</code>	Stable step identifier used for feedback and possible replan joins.
<code>steps[].name</code>	Skill name selected from the allowed registry.
<code>steps[].args</code>	Skill arguments. They should refer to known entity IDs or explicit primitive values.
<code>steps[].requires</code>	Dependencies between steps, if any.
<code>steps[].on_failure</code>	Recovery policy proposed by planner_llm for this step. If the step fails, nao_orchestrator uses this field to decide whether to fail, continue, ask the user, request clarification, or send feedback back to planner_llm for replanning. Allowed values include replan , clarify , ask_user , continue , ignore , and fail .

Runtime recovery metadata. The current contract still supports recovery metadata beyond the compact table. Each step can carry **retry_budget**, and the plan envelope can carry a plan-level **retry_budget**. The allowed failure policy is not a separate **retry** label; it is expressed through **on_failure** values such as **replan**, **clarify**, **ask_user**, or **fail**. The plan envelope can also carry **communication_policy**, which controls whether acknowledgement, progress, completion, or failure messages should be emitted by the planner/dialogue path. These fields are useful runtime controls, but they are secondary to the ordered **steps** list in the minimal explanation.

Runtime recovery metadata. The current contract supports recovery metadata as skill failure may require controlled recovery. Each step may carry a **retry_budget**, and the plan envelope may also carry a plan-level **retry_budget**. This does not mean that **retry** is a separate **on_failure** value. Instead, retry behaviour is controlled by **nao_orchestrator** using the available retry budget together with the step’s **on_failure** policy. The plan envelope may also carry **communication_policy**. This controls when the planner/dialogue path should communicate progress, completion, clarification needs, or failure.

When a step fails with **on_failure = replan**, **nao_orchestrator** halts or blocks continuation, publishes execution feedback to **planner_llm**, and waits for a revised validated plan before dispatching further actions. This keeps execution authority inside **nao_orchestrator**, while allowing recovery to be driven by planner supervision rather than by hard-coded local

replanning.

These fields are included because they support runtime recovery, traceability, and human-robot interaction, but the essential planner output remains the ordered **steps** list. If the planner cannot produce a plan, the output should represent that as a planning result rather than filling the plan with ambiguous failure fields. For the simple version, **steps** is the most important field.

11 Contract 5: Execution Feedback

For the simple version, execution feedback can be minimal. The planner needs to know which goal is affected, which step failed or succeeded, and a compact result if that result is useful for replanning or dialogue. A larger payload is justified only when the planner supervision logic, orchestrator, traces, or debug tooling use the extra fields.

```
{
  "goal_id": "goal_turn_123",
  "status": "failed",
  "step": {
    "id": "step_1",
    "name": "look_at"
  },
  "reason": "target frame unavailable",
  "result_payload": {
    "target_found": false,
    "target_kind": "person"
  }
}
```

Table 6: Fields in Contract F: minimal planner-relevant feedback.

Field	Meaning
<code>goal_id</code>	User objective affected by the feedback.
<code>status</code>	Outcome of the executed step or plan segment.
<code>step.id</code>	Step identifier from the planner output. This is the key field for replanning.
<code>step.name</code>	Skill name that produced the feedback.
<code>reason</code>	Short explanation of the outcome. For failures, this should explain what prevented the step from completing.
<code>result_payload</code>	Optional structured evidence that the planner can use for replanning or clarification.

Runtime feedback may include additional fields used by `nao_orchestrator`, logging, and planner supervision. Examples include `plan_id`, `plan_version`, `event_type`, `validation_status`, `blocking`, `needs_user_input`, `timestamp_sec`, `result_summary`, and retry metadata. These

fields are useful for execution control and debugging, but the minimal planner-facing feedback is the goal, step, status, reason, and result payload.

12 Contract 6: Planner Dialogue Act

Planner dialogue acts represent planner intent to communicate, not direct speech execution. They are relayed through the orchestrator and realized by the dialogue layer.

```
{
  "goal_id": "goal_turn_123",
  "act": "ask_clarification",
  "await_user_response": true,
  "reason": "multiple visible people",
  "text_hint": "Which person should I look at?",
  "slots_needed": ["target_person"]
}
```

Allowed dialogue acts are acknowledgement, progress update, clarification, help request, failure explanation, completion notification, and cancellation notification. The planner does not speak directly; it emits a structured communication request.